

DBMS – II UNIT

Two marks questions:

1. What is the degree of a relation? What is the degree of the following relation?

STUDENT(Name, SSn, Home_phone, Address, Office_phone, Age, Gpa)

The total number of attributes in a relation is known as the degree of the relation. This is also called as 'arity'.

The **STUDENT** relation defined above has degree 7.

2. Define Cartesian product.

A Cartesian product is the result of joining every row in one table with every row in another table. This occurs when there is no WHERE clause to restrict rows.

The Cartesian product specifies all possible combinations of values from the underlying domains. Hence, if we denote the total number of values, or cardinality, in a domain D by |D| (assuming that all domains are finite), the total number of tuples in the Cartesian product is $|\text{dom}(A1)| \times |\text{dom}(A2)| \times \dots \times |\text{dom}(A_n)|$.

3. What is relation schema? List the characteristics of Relations?

The relation schema describes the column heads for the table. The schema specifies the relation's name, the name of each field (column, attribute) and the 'domain' of each field.

Characteristics of Relations:

- Tuples in a relation do not have any specified order.
- Ordering of values within a tuple is significant.
- Each tuple in a relation contains a single value for each of its attribute.
- No two tuples are identical in a relation.

4. Define a) tuple b) Attribute c) Domain

Tuple : A tuple is a single row in a database that contains a single record for such a relation.

A row or record is another name for a single item in a table, which is what a triple is.

Attribute : Attributes are the properties which define a relation. For example, in a customer database, the attributes might be name, address, and phone number.

Domain : A domain D is a set of atomic values. Each simple attribute of entity type is associated with domain of values which specifies set of values that can be assigned to that attribute for individual entity.

5. List any two categories of relational model constraints?

- Domain constraints.
- Key constraints.
- Entity Integrity constraints.
- Referential integrity constraints.

6. Define super key. Give an example.

Super key is an attribute set that can uniquely identify a tuple. It is a set of attributes in which no two tuples can have the same combination of values for these attributes. A super key is a superset of a candidate key.

For example: In the EMPLOYEE table, for (EMPLOYEE_ID, EMPLOYEE_NAME), the name of two employees can be the same, but their EMPLOYEE_ID can't be the same. Hence, this combination can be a key.

7. What is candidate key? Give an example.

A candidate key is a subset of the super key that can uniquely identify the other attributes of the table. The candidate key helps in determining the prime and non-prime attributes of a table and ensures the integrity of the data by preventing duplicate data.

For example: In the Student table, for (Roll_No, ID, Name), Roll_No and ID are candidate keys as they will contain unique values only.

8. What is entity integrity constraint and referential entity constraint?

The entity integrity constraint states that no primary key value can be NULL. This is because the primary key value is used to identify individual tuples in a relation. Having null values for the primary key implies that we cannot identify some tuples.

A referential integrity constraint is specified between two tables and is used to maintain the consistency among tuples of the two tables. It states that 'a tuple in one relation that refers to another relation must refer to an existing tuple in that relation'.

9. Mention the properties of key in a relational schema.

- Two distinct tuples in any state of the relation cannot have identical values for (all) the attributes in the key.
- It is a minimal superkey from which we cannot remove any attributes and still have the uniqueness constraint in condition 1 hold.

10. What is entity integrity constraint and referential integrity constraint?

The entity integrity constraint states that no primary key value can be NULL. This is because the primary key value is used to identify individual tuples in a relation. Having null values for the primary key implies that we cannot identify some tuples.

A referential integrity constraint is specified between two tables and is used to maintain the consistency among tuples of the two tables. It states that 'a tuple in one relation that refers to another relation must refer to an existing tuple in that relation'. The widely used referential constraint is foreign key.

11. Give any two basic operations that can change the state of relations in the database.

(i) Insert: This operation is performed to add a new tuple in a relation.

(ii) Delete: This operation is carried out to remove a tuple from a relation.

12. What is the function of Insert, update and delete functions?

Insert: This operation is performed to add a new tuple in a relation.

Delete: This operation is carried out to remove a tuple from a relation.

Update: This operation is used to update or modify existing values of record in a data table.

13. Mention any two update anomalies.

(i) **Insertion Anomaly:** If there is a new row inserted in the table and it creates the inconsistency in the table then it is called the insertion anomaly.

(ii) **Deletion Anomaly:** If we delete some rows from the table and if any other information or data which is required is also deleted from the database, this is called the deletion anomaly in the database.

14. State interpretations of NULL values.

- The value unknown (value exists but is not known)
- Value not available (exists but is purposely withheld)
- Attribute not applicable (undefined for this tuple)

15. Define functional dependency.

A functional dependency is a constraint between two sets of attributes typically between the primary key and other non-key attributes. Functional dependency is when one attribute determines the value of another attribute. It typically exists between the primary key and non-key attribute within a table.

FD are expressed as $X \rightarrow Y$, where X is the determinant and Y is the dependent.

Eg :

Assume we have an employee table with attributes: Emp_Id, Emp_Name, Emp_Address.

Here, Emp_Id attribute can uniquely identify the Emp_Name attribute.

Functional dependency can be written as $\text{Emp_Id} \rightarrow \text{Emp_Name}$.

16. What do mean by “normalization of data”?

Normalization is the process of decomposing the relations into well structured relations. It is a process used to organize the data in the database and to remove redundancy of data, insertion anomaly, update anomaly and deletion anomaly.

Unsatisfactory relation schemas that do not meet certain conditions are decomposed into smaller relation schemas that meet the tests and hence possess the desirable properties.

17. What is 1NF,2NF,3NF?

1NF (First Normal Form) : A relation is said to be in 1NF if all the attributes of the relation are atomic and single valued.

2NF (Second Normal Form) : A relation is said to be in 2NF if it is first in 1NF and no partial dependency exists.

3NF (Third Normal Form) : A relation is said to be in 3NF if it is first in 2NF and no transitive functional dependency exists.

18. What is BCNF?

BCNF (Boyce Codd Normal Form) is the advanced version of 3NF. BCNF is stricter than 3NF. A relation is said to be in BCNF if it is first in 3NF and every key is a candidate key. For any dependency $A \rightarrow B$, A should be a superkey.

Eg:

Student	Teacher	Subject
Jhansi	P. Naresh	DBMS
Jhansi	K.Das	C
Subbu	P. Naresh	DBMs
Subbu	R. Prasad	C

Teacher-> subject violates BCNF [since teacher is not a candidate key].

Table can be decomposed as: Teacher	Subject
P. Naresh	DBMS
K.Das	C
R. Prasad	C

Student	Teacher
Jhansi	P. Naresh
Jhansi	K.Das
Subbu	P. Naresh
Subbu	R. Prasad

19. What is equivalence sets of functional dependencies?

Two or more than two sets of functional dependencies are called equivalence if the right-hand side of one set of functional dependency can be determined using the second FD set, similarly the right-hand side of the second FD set can be determined using the first FD set.

Two sets of functional dependencies E and F are equivalent if $E^+ = F^+$.

20. What is minimal sets of functional dependencies?

A minimal cover of a set of functional dependencies E is a set of functional dependencies that satisfies the property that every dependency in E is in the closure F^+ of F.

F to be minimal if it satisfies the following conditions:

1. Every dependency in F has a single attribute for its right-hand side.
2. We cannot replace any dependency $X \rightarrow A$ in F with a dependency $Y \rightarrow A$, where Y is a proper subset of X, and still have a set of dependencies that is equivalent to F.
3. We cannot remove any dependency from F and still have a set of dependencies that is equivalent to F.

21. Define atomic value with respect to tuple.

Each value in a tuple is an atomic value; By atomic value, we mean that each value in the tuple is indivisible as far as the relational model is concerned.

22. What are flat relational models?

Each value in a tuple is an atomic value; that is, it is not divisible into components within the framework of the basic relational model. Hence, composite and multivalued attributes are not allowed. This model is sometimes called the flat relational model.

23. Define model-based constraints.

Constraints that are inherent in the data model. We call these inherent model-based constraints or implicit constraints.

24. Define schema-based constraints.

Constraints that can be directly expressed in schemas of the data model, typically by specifying them in the DDL. We call these schema-based constraints or explicit constraints.

25. What are application-based constraints?

Constraints that cannot be directly expressed in the schemas of the data model, and hence must be expressed and enforced by the application programs. We call these application-based or semantic constraints or business rules.

26. What are component values of relation?

27. What does domain constraints specify?

Domain constraints specify that within each tuple, the value of each attribute must be an atomic value from the domain $\text{dom}(A)$. It means composite and multi-valued attributes are not allowed.

28. Define the term minimal superkey.

Minimal super-key is also called as Key. It is defined as that part of super-key from which any attribute cannot be removed without sacrificing the uniqueness constraint.

3 OR MORE MARKS

1. Discuss Domains and attributes with respect to relational scheme?

Relation schema defines the design and structure of the relation. It consists of the relation name, set of attributes/field names/column names. Every attribute would have an associated domain.

Domain: Each simple attribute of entity type is associated with domain of values which specifies set of values that can be assigned to that attribute for individual entity. For example, If the range of ages allowed for employees is between 20 and 60, we can specify domain for age to be set of integers between 20 and 70.

A data type or format is also specified for each domain. For example, The data type for Employee_ages is an integer number between 15 and 80. A domain is thus given a name, data type, and format.

Attribute: It contains the name of a column in a particular table. Each attribute A_i must have a domain, $\text{dom}(A_i)$.

A relation schema R , denoted by $R(A_1, A_2, \dots, A_n)$, is made up of a relation name R and a list of attributes, $A_1, A_2, \dots, A_n$. The degree (or arity) of a relation is the number of attributes of its relation schema. A relation of degree five, which stores information about students, would contain five attributes describing each student as follows:

`STUDENT(RollNo, Name, Home_phone, Address, Age)`

For this relation schema, STUDENT is the name of the relation, which has five attributes. We can specify previously defined domains for some of the attributes of the STUDENT relation:

`dom(RollNo) = rollno; dom(Name) = Names; dom(HomePhone) = phone_no;`

2. Discuss tuples and relations with respect to relational scheme?

Relation: A relation is nothing but a table of values. A table has rows and columns, where rows represent records and columns represent the attributes.

A finite set of tuples in the relational database system represents relation instance. A relation schema describes the relation name (table name), attributes, and their names.

Tuple: A single row of a table, which contains a single record for that relation is called a tuple.

A relation r of the relation schema $R(A_1, A_2, \dots, A_n)$, also denoted by $r(R)$, is a set of n -tuples. Each n -tuple t is an ordered list of n values, where each value is an element of $\text{dom}(A_i)$ or is a special NULL value.

Eg:

RollNo	Name	PhoneNo	Address	Age
12	Benjamin	NULL	Mangalore	19
35	David	9895543210	Udupi	19
05	Rohan	8956472350	Bangalore	18

Given above is a STUDENT relation. Each tuple in the relation represents a particular student entity. The relation is displayed as a table where each tuple is shown as a row.

3.Explain the characteristics of Relations?

Ordering of tuples in a relation: Since a relation is a set of tuples and a set has no particular order among its elements, hence, tuples in a relation do not have any specified order. However, tuples in a relation can be logically ordered by the values of various attributes. In that case, information in a relation remains same, only the order of tuple varies. Hence, tuple ordering in a relation is irrelevant.

Ordering of values within a tuple: An n-tuple is an ordered set of attribute values that belongs to the domain D, so, the order in which the values appear in the tuples is significant. However, if a tuple is defined as a set of (<attribute> : <value>) pairs, the order in which attributes appear in a tuple is irrelevant. This is due to the reason that there is no preference for one attribute value over another.

Values and nulls in the tuples: Relational model is based on the assumption that each tuple in a relation contains a single value for each of its attribute. Hence, a relation does not allow composite and multivalued attributes. Moreover, it allows denoting the value of the attribute as null, if the value does not exist for that attribute or the value is unknown.

No two tuples are identical in a relation: Since a relation is a set of tuples and a set does not have identical elements. Therefore, each tuple in a relation must be uniquely identified by its contents. In other words, two tuples with the same value for all the attributes (that is, duplicate tuples) cannot exist in a relation.

Interpretation of a relation: A relation can be used to interpret facts about entities as a type of assertion. A relation can also be used to interpret facts about relationships.

4. Explain the categories of constraints?

(i) Domain Constraint

Domain constraint defines the domain or set of values for an attribute . It specifies that the value taken by the attribute must be the atomic value from its domain. The data type of domain includes string, character, integer, time, date, currency, etc.

Eg:

ID	NAME	SEMENSTER	AGE
1000	Tom	1 st	17
1001	Johnson	2 nd	24
1002	Leonardo	5 th	21
1003	Kate	3 rd	19
1004	Morgan	8 th	A

Not allowed. Because AGE is an integer attribute

(ii) Key Constraint

An attribute that can uniquely identify a tuple in a relation is called the key of the table. It ensures that data entered by the user into columns must be within the criteria specified by the condition.

An entity set can have multiple keys, but out of which one key will be the primary key. All the values of primary key must be unique. The value of primary key must not be null, hence Not Null constraint is also a part of key constraint.

Eg:

ID	NAME	SEMENSTER	AGE
1000	Tom	1 st	17
1001	Johnson	2 nd	24
1002	Leonardo	5 th	21
1003	Kate	3 rd	19
1002	Morgan	8 th	22

Not allowed. Because all row must be unique

(iii) Entity Integrity Constraint

The entity integrity constraint states that primary key value can't be null. This is because the primary key value is used to identify individual rows in relation and if the primary key has a null value, then we can't identify those rows.

A table can contain a null value other than the primary key field.

Eg: **EMPLOYEE**

EMP_ID	EMP_NAME	SALARY
123	Jack	30000
142	Harry	60000
164	John	20000
	Jackson	27000

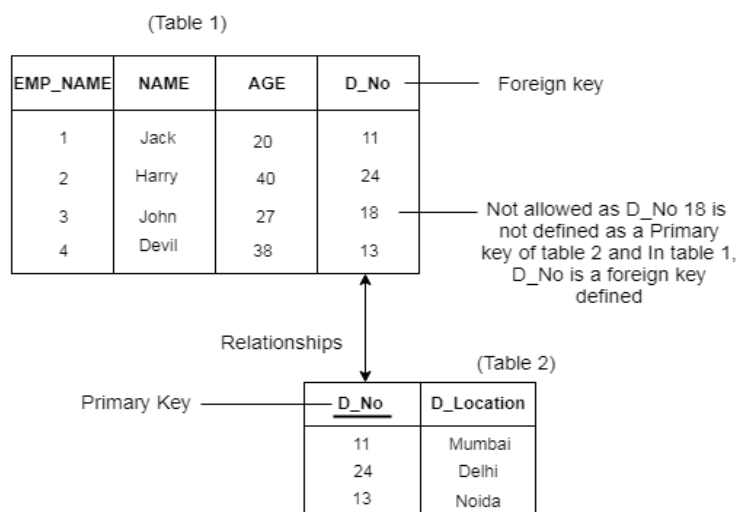
Not allowed as primary key can't contain a NULL value

(iv) Referential Integrity Constraints

A referential integrity constraint is specified between two tables and is used to maintain the consistency among tuples of the two tables. It states that 'a tuple in one relation that refers to another relation must refer to an existing tuple in that relation'.

If a foreign key in Table 1 refers to the Primary Key of Table 2, then every value of the Foreign Key in Table 1 must be null or be available in Table 2.

Eg:



5. Write a note on key constraints?

An attribute that can uniquely identify a tuple in a relation is called the key of the table. It ensures that data entered by the user into columns must be within the criteria specified by the condition.

We have 5 types of key constraints in DBMS.

(i) Unique

Sometimes we need to maintain only unique data in the column of a database table, this is possible by using a unique constraint. Unique constraint ensures that all values in a column are unique.

Eg:

```
CREATE TABLE Persons (  
    ID int UNIQUE,  
    Name varchar(255) NOT NULL,  
    Age int,  
);
```

We are not supposed to enter the data that is already present in ID column.

(ii) Check

Check constraint ensures that the data entered by the user for that column is within the range of values or possible values specified.

Eg:

```
CREATE TABLE STUDENT (  
    ID int ,  
    Name varchar(255) ,  
    Age int,  
    CHECK (Age<=18)  
);
```

the only possible values that the age column will accept is [0 -17].

(iii) Not Null

Once not null is applied to a particular column, you cannot enter null values to that column and restricted to maintain only some proper value other than null. A not-null constraint cannot be applied at table level.

Eg:

```
CREATE TABLE Orders (  
    OrderID int NOT NULL,  
    OrderNumber int NOT NULL,  
    PersonID int,  
    PRIMARY KEY (OrderID)  
);
```

We cannot insert a null value to OrderID and OrderNumber columns.

(iv) Primary Key

A primary key is a constraint in a table that uniquely identifies each row record in a database table. A particular column is made as a primary key column by using the primary key keyword followed with the column name.

Eg:

```
CREATE TABLE EMP (  
    ID INT  
    NAME VARCHAR (20)  
    AGE INT  
    PRIMARY KEY (ID)  
);
```

Here we have used the primary key on ID column then ID column must contain unique values.

(v) Foreign Key

The foreign key a constraint is a column or list of columns that points to the primary key column of another table.

Eg

Reference Table:

```
CREATE TABLE CUSTOMERS1 (  
    ID INT ,  
    NAME VARCHAR (20) ,  
    COURSE VARCHAR(10) ,  
    PRIMARY KEY (ID)  
);
```

Child Table:

```
CREATE TABLE CUSTOMERS2 (  
    ID INT ,  
    MARKS INT,  
    REFERENCES CUSTOMERS1 (ID)  
);
```

6. Discuss Entity integrity and referential integrity constraints?

- **Entity Integrity Constraint**

The entity integrity constraint states that primary key value can't be null. This is because the primary key value is used to identify individual rows in relation and if the primary key has a null value, then we can't identify those rows.

A table can contain a null value other than the primary key field.

Eg:

EMPLOYEE

EMP_ID	EMP_NAME	SALARY
123	Jack	30000
142	Harry	60000
164	John	20000
	Jackson	27000

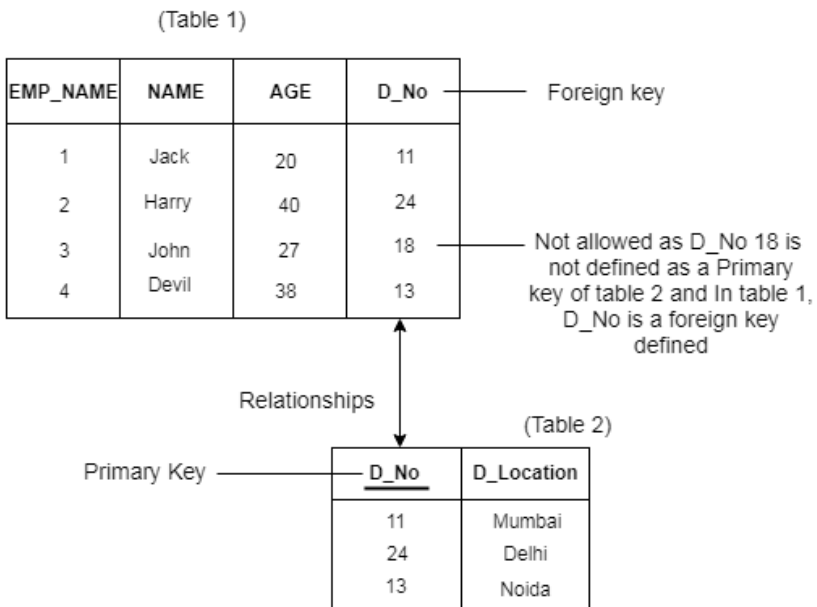
Not allowed as primary key can't contain a NULL value

- **Referential Integrity Constraint**

A referential integrity constraint is specified between two tables and is used to maintain the consistency among tuples of the two tables. It states that ‘a tuple in one relation that refers to another relation must refer to an existing tuple in that relation’.

If a foreign key in Table 1 refers to the Primary Key of Table 2, then every value of the Foreign Key in Table 1 must be null or be available in Table 2.

Eg:

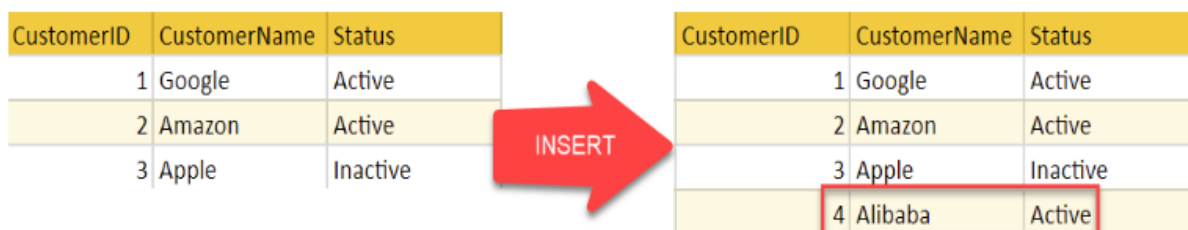


7. Explain the three basic operations that can change the states of relations in the database?

(i) Insert: This operation is performed to add a new tuple in a relation. The insert operation is capable of violating constraints.

Syntax: `INSERT INTO table_name (column1, column2, column3, ...) VALUES (value1, value2, value3, ...);`

Eg:



(ii) Delete: This operation is carried out to remove a tuple from a relation. A particular data record from a table can be removed by performing such operation. The delete operation can violate only referential integrity, if tuple being removed is referenced by foreign key from other tuples in the database.

Syntax: `DELETE FROM table [WHERE conditions];`

Eg:

CustomerID	CustomerName	Status
1	Google	Active
2	Amazon	Active
3	Apple	Active
4	Alibaba	Active



CustomerID	CustomerName	Status
1	Google	Active
2	Amazon	Active
4	Alibaba	Active

(iii) Modify: The operation aims at causing a change in the values of some attributes in existing tuples. This is useful in modifying existing values of record in a data table. Usually, this operation does not cause problems only if the modification is directed on neither primary key nor foreign key. Whenever applied, these operations must enforce integrity constraints specified on relational database schema.

Syntax: `UPDATE table
SET column1 = expression1,
column2 = expression2,
column_n = expression_n
[WHERE conditions];`

Eg:

CustomerID	CustomerName	Status
1	Google	Active
2	Amazon	Active
3	Apple	Inactive
4	Alibaba	Active



CustomerID	CustomerName	Status
1	Google	Active
2	Amazon	Active
3	Apple	Active
4	Alibaba	Active

8. Which are the different update anomalies? Discuss.

Anomaly means the inconsistency occurred in the relational table during the operations performed on the relational table.

Eg:

Worker_id	Worker_name	Worker_dept	Worker_address
65	Ramesh	ECT001	Jaipur
65	Ramesh	ECT002	Jaipur
73	Amit	ECT002	Delhi
79	Rajesh	ECT669	Mumbai

The above table is not normalized, and there is definitely a chance of anomalies present in the table.

Insertion Anomaly

If there is a new row inserted in the table and it creates the inconsistency in the table then it is called the insertion anomaly.

Eg : If we create a new row of a worker, and if it is not allocated to any department then we cannot insert it in the table so, it will create an insertion anomaly.

Deletion Anomaly

If we delete some rows from the table and if any other information or data which is required is also deleted from the database, this is called the deletion anomaly in the database.

Eg : If we want to delete the department number ECT669 then the details of Rajesh will also be deleted since Rajesh's details are dependent on the row of ECT669.

Modification Anomaly

When we change the value of one attribute of a row in the table, and if it leads to the inconsistency of the table then this anomaly occurs.

Eg: If we want to update the address of Ramesh then we will have to update all the rows where Ramesh is present.

9. Can tuples have null values? Discuss.

In some schema designs, we may group many attributes together into a one relation. If many of the attributes do not apply to all tuples in the relation, we end up with many NULLs in those tuples. This can waste space at the storage level and may also lead to problems with understanding the meaning of the attributes.

Another problem with NULLs is how to account for them when aggregate operations such as COUNT or SUM are applied. As SELECT and JOIN operations involve comparisons, if NULL values are present, the results may become unpredictable. Moreover, NULLs can have multiple interpretations, such as the following:

- The attribute does not apply to this tuple. For example, Visa_status may not apply to U.S. students.
- The attribute value for this tuple is unknown. For example, the Date_of_birth may be unknown for an employee.
- The value is known but absent; that is, it has not been recorded yet. For example, the Phone_Number for an employee may exist, but may not be available and recorded yet.

Having the same representation for all NULLs compromises the different meanings they may have.

10. Discuss functional dependencies with example?

A functional dependency is a constraint between two sets of attributes from the database.

Functional Dependency is when one attribute determines the value of another attribute i.e, if one data item value is known, then it is possible to determine the other data value.

FD are expressed as $X \rightarrow Y$, where X is the determinant and Y is the dependent.

Eg:

EMP_ID	EMP_NAME	SALARY
123	Jack	30000
142	Harry	60000
164	John	20000

In this example, if we know the value of Employee id, we can obtain Employee Name and Salary. We can say that the Employee Name and Salary are functionally dependent on Employee id.

Types:

- **Trivial functional dependency**

In Trivial Functional Dependency, a dependent is always a subset of the determinant.
i.e. If $X \rightarrow Y$ and Y is the subset of X, then it is called trivial functional dependency.

Eg:

Roll_no	Name	Age
42	Abc	17
43	Pqr	18
44	xyz	18

Here, $\{\text{roll_no, name}\} \rightarrow \text{name}$ is a trivial functional dependency, since the dependent **name** is a subset of determinant set $\{\text{roll_no, name}\}$

Similarly, $\text{roll_no} \rightarrow \text{roll_no}$ is also an example of trivial functional dependency.

- **Non-trivial functional dependency**

In Non-trivial functional dependency, the dependent is strictly not a subset of the determinant.
i.e. If $X \rightarrow Y$ and Y is not a subset of X, then it is called Non-trivial functional dependency.

Eg:

Roll_no	Name	Age
42	Abc	17
43	Pqr	18
44	xyz	18

Here, $\text{roll_no} \rightarrow \text{name}$ is a non-trivial functional dependency, since the dependent **name** is not a subset of determinant **roll_no**.

- **Multivalued dependency**

In Multivalued dependency, entities of the dependent set are not dependent on each other.
i.e. if $a \rightarrow \{b, c\}$ and there exists no functional dependency between b and c, then it is called a multivalued functional dependency. Eg:

Roll_no	Name	Age
42	Abc	17
43	Pqr	18
44	xyz	18

Here, $\text{roll_no} \rightarrow \{\text{name, age}\}$ is a multivalued functional dependency, since the dependents name & age are not dependent on each other

• Transitive dependency

An attribute is transitively dependent, if its value is determined by another attribute which is not a key.

i.e, If $a \rightarrow b$ & $b \rightarrow c$, then $a \rightarrow c$. This is a transitive functional dependency.

Eg:

Roll_no	Name	Dept	Building_No
42	Abc	CO	4
43	Pqr	EC	2
44	xyz	IT	1

Here, $\text{enrol_no} \rightarrow \text{dept}$ and $\text{dept} \rightarrow \text{building_no}$.

Hence, $\text{enrol_no} \rightarrow \text{building_no}$ is a valid FD.

This is transitive dependency.

11. Write a note on “normalization of relations”.

- Normalization is the process of organizing the data in the database.
- Normalization is used to minimize the redundancy from a relation or set of relations. It is also used to eliminate undesirable characteristics like Insertion, Update, and Deletion Anomalies.
- Normalization divides the larger table into smaller and links them using relationships.
- The normal form is used to reduce redundancy from the database table.
- The main reason for normalizing the relations is removing these anomalies. Failure to eliminate anomalies leads to data redundancy and can cause data integrity and other problems as the database grows.
- Normalization consists of a series of guidelines that helps to guide you in creating a good database structure.

12. Discuss 1NF with example.

- A relation will be 1NF (First Normal Form) if it contains an atomic value.
- It states that an attribute of a table cannot hold multiple values. It must hold only single-valued attribute.
- First normal form disallows the multi-valued attribute, composite attribute, and their combinations.

Consider the following example:

Employee

Emp_id	Emp_name	Emp_address	Emp_mobile
101	Herschel	New Delhi	8912312390
102	Jon	Kanpur	8812121212 9900012222
103	Ron	Chennai	7778881212
104	Lester	Bangalore	9990000123 8123450987

Here, the Relation employee is not in 1NF, because employees with employee id 102 and 104 are having two phone numbers. i.e. the Emp_mobile attribute is a multi valued attribute.

After normalization to 1NF the relation will be like this:

Emp_id	Emp_name	Emp_address	Emp_mobile
101	Herschel	New Delhi	8912312390
102	Jon	Kanpur	8812121212
102	Jon	Kanpur	9900012222
103	Ron	Chennai	7778881212
104	Lester	Bangalore	9990000123
104	Lester	Bangalore	8123450987

13. Discuss 2NF with example.

The second normal form is based on the full functional dependency. A relation is said to be in 2NF if it is first in 1NF and all non-key attributes are fully functional dependent on the primary key.i.e. no partial dependency exists.

Second Normal Form applies to relations with composite keys, that is, relations with a primary key composed of two or more attributes.

Consider the following example:

TABLE_PURCHASE_DETAIL

Customer ID	Store ID	Purchase Location
1	1	Los Angeles
1	3	San Francisco
2	1	Los Angeles
3	2	New York
4	3	San Francisco

This table has a composite primary key {Customer ID, Store ID}. The non-key attribute is Purchase Location. In this case, Purchase Location only depends on Store ID, which is a part of the primary key. Therefore, this table does not satisfy second normal form.

To bring this table to second normal form, we have to break the table into two tables as shown below:

TABLE_PURCHASE		TABLE_STORE	
Customer ID	Store ID	Store ID	Purchase Location
1	1	1	Los Angeles
1	3	2	New York
2	1	3	San Francisco
3	2		
4	3		

Now, in the table TABLE_STORE, the column Purchase Location is fully dependent on the primary key of that table, which is Store ID.

14. Discuss 3NF with example.

- A relation will be in 3NF if it is in 2NF and not contain any transitive partial dependency.
- 3NF is used to reduce the data duplication. It is also used to achieve the data integrity.
- If there is no transitive dependency for non-prime attributes, then the relation must be in third normal form.

A relation is in third normal form if it holds atleast one of the following conditions for every non-trivial function dependency $X \rightarrow Y$.

1. X is a super key.
2. Y is a prime attribute, i.e., each element of Y is part of some candidate key.

Consider the following example:

TABLE_BOOK_DETAIL			
Book ID	Genre ID	Genre Type	Price
1	1	Gardening	25.99
2	2	Sports	14.99
3	1	Gardening	10.00
4	3	Travel	12.99
5	2	Sports	17.99

In the above table, Book ID determines Genre ID, and Genre ID determines Genre Type. Therefore, Book ID determines Genre Type via Genre ID and we have transitive functional dependency, and this structure does not satisfy third normal form.

To bring this table to 3NF, we split the table into two as follows:

TABLE_BOOK			TABLE_GENRE	
Book ID	Genre ID	Price	Genre ID	Genre Type
1	1	25.99	1	Gardening
2	2	14.99	2	Sports
3	1	10.00	3	Travel
4	3	12.99		
5	2	17.99		

Now all non-key attributes are fully functional dependent only on the primary key. In TABLE_BOOK, both Genre ID and Price are only dependent on Book ID. In TABLE_GENRE, Genre Type is only dependent on Genre ID.

15. Discuss BCNF with example.

BCNF (Boyce Codd Normal Form) is the advanced version of 3NF. A table is in BCNF if every functional dependency $X \rightarrow Y$, X is the super key of the table. For BCNF, the table should be in 3NF, and for every FD, LHS is super key.

Example

Consider a relation R with attributes (student, subject, teacher).

Student	Teacher	Subject
Jhansi	P.Naresh	Database
jhansi	K.Das	C
subbu	P.Naresh	Database
subbu	R.Prasad	C

```
F: { (student, Teacher) -> subject
      (student, subject) -> Teacher
      Teacher -> subject }
```

Candidate keys are (student, teacher) and (student, subject).

The above relation is in 3NF [since there is no transitive dependency]. A relation R is in BCNF if for every non-trivial FD $X \rightarrow Y$, X must be a key.

The above relation is not in BCNF, because in the FD (teacher->subject), teacher is not a key.

Decomposition for BCNF

Teacher-> subject violates BCNF [since teacher is not a candidate key].

If $X \rightarrow Y$ violates BCNF then divide R into $R_1(X, Y)$ and $R_2(R-Y)$.

So R is divided into two relations $R_1(\text{Teacher, subject})$ and $R_2(\text{student, Teacher})$.

R1

Teacher	Subject
P.Naresh	database
K.DAS	C
R.Prasad	C

R2

Student	Teacher
Jhansi	P.Naresh
Jhansi	K.Das
Subbu	P.Naresh
Subbu	R.Prasad

16. Consider the universal relation $R = \{A, B, C, D, E, F, G, H, I, J\}$ and the set of functional dependencies $F = \{ \{A, B\} \rightarrow \{C\}, \{A\} \rightarrow \{D, E\}, \{B\} \rightarrow \{F\}, \{F\} \rightarrow \{G, H\}, \{D\} \rightarrow \{I, J\} \}$. What is the key for R? Decompose R into 2NF and then 3NF relations.

Solution:

$A \rightarrow DE$ (given) $\Rightarrow A \rightarrow D$ and $A \rightarrow E$

Since $A \rightarrow D$ and $D \rightarrow IJ$ (given) $\Rightarrow A \rightarrow IJ$

Using the union rule $A \rightarrow ADEIJ$, thus $AB \rightarrow ABDEIJ$ (augmentation)

Also $AB \rightarrow C$ (given) $\Rightarrow AB \rightarrow ABCDEIJ$.

Since $B \rightarrow F$ (given) and $F \rightarrow GH$ (given), $B \rightarrow GH$ (transitivity)

Thus $AB \rightarrow AGH$ holds. Also $AB \rightarrow AF$ holds from $B \rightarrow F$ (given)

Finally, using the union rule $AB \rightarrow ABCDEFGHIJ$.

So AB is a key. This can also be determined by calculating AB^+ with respect to the set F.

2NF

R1 (A, B, C)

R2 (A, D, E, I, J)

R3 (B, F, G, H)

3NF

R1 (A, B, C)

R2.1 (A, D, E) R2.2 (D, I, J)

R3.1 (B, F) R3.2 (F, G, H)

17. Consider a relation $R(A, B, C, D, E)$ with the following dependencies: $AB \rightarrow C$, $CD \rightarrow E$, $DE \rightarrow B$. Is AB a candidate key of this relation? If not, is ABD? Explain your answer.

18. Given below are two sets of FDs for a relation $R(A, B, C, D, E)$ are they equivalent?

(i) $A \rightarrow B$, $AB \rightarrow C$, $D \rightarrow AC$, $D \rightarrow E$

(ii) $A \rightarrow BC$, $D \rightarrow AE$

Let $F1 = \{ A \rightarrow B, AB \rightarrow C, D \rightarrow AC, D \rightarrow E \}$

$F2 = \{ A \rightarrow BC, D \rightarrow AE \}$

FDs of $F1$ are implied in $F2$ as follows:

In $F2$, $A^+ = ABC$ (hence FDs of $F1$ $A \rightarrow B$ and $AB \rightarrow C$ both are implied in $F2$)

$D^+ = ABCDE$ (hence FDs of $F1$ $D \rightarrow E$ and $D \rightarrow AC$ both are implied in $F2$)

FDs of $F2$ are implied in $F1$ as follows:

In $F1$ $A^+ = ABC$ (hence FD of $F2$ $A \rightarrow BC$ implied in $F1$)

$D^+ = ABCDE$ (hence FD of $F2$ $D \rightarrow AE$ implied in $F1$)

Hence $F1$ equivalent to $F2$.

19. A set of FDs for the relation $R \{A, B, C, D, E, F\}$ is $AB \rightarrow C, C \rightarrow A, BC \rightarrow D, ACD \rightarrow B, BE \rightarrow C, EC \rightarrow FA, CF \rightarrow BD, D \rightarrow E$. Find a minimum cover for this set of FDs.

20. Prove that any relation schema with two attributes is in BCNF.

If there is a relation schema with two attributes say $R(A,B)$ then if either of the attribute becomes the key let say A be the key, then $A \rightarrow B$ is true. So R is in BCNF.

Otherwise both of them is the key then $\{A,B\} \rightarrow \{A,B\}$, so BCNF holds.

Proof:

Let a, b be the two attributes in a relation R . The possible functional dependencies are:

Case 1: LHS contains both attributes

$a b \rightarrow \text{---}$

Clearly this is a trivial functional dependency because RHS attributes form subset of LHS attributes .

Case 2 : LHS contains only one attribute.

$a \rightarrow \text{---}$

$b \rightarrow \text{---}$

Clearly in this case, the LHS attribute will be candidate key.

Hence the possible functional dependencies are either trivial or the LHS attributes form a superkey. So, the relation having only two attributes will be in BCNF.

21. Prove and disprove the following inference rules for functional dependencies.

The following six rules IR1 through IR6 are well-known inference rules for functional dependencies:

IR1 (reflexive rule)1: If $X \supseteq Y$, then $X \rightarrow Y$.

Eg: $X = \{a, b, c, d, e\}$
 $Y = \{a, b, c\}$

IR2 (augmentation rule)2: $\{X \rightarrow Y\} \models XZ \rightarrow YZ$.

Eg: For $R(ABCD)$, if $A \rightarrow B$ then $AC \rightarrow BC$

IR3 (transitive rule): $\{X \rightarrow Y, Y \rightarrow Z\} \models X \rightarrow Z$.

IR4 (decomposition, or projective, rule): $\{X \rightarrow YZ\} \models X \rightarrow Y$.

Proof:

1. $X \rightarrow Y$ (given)
2. $X \rightarrow Z$ (given)
3. $X \rightarrow XY$ (using IR2 on 1 by augmentation with X. Where $XX = X$)
4. $XY \rightarrow YZ$ (using IR2 on 2 by augmentation with Y)
5. $X \rightarrow YZ$ (using IR3 on 3 and 4)

IR5 (union, or additive, rule): $\{X \rightarrow Y, X \rightarrow Z\} \models X \rightarrow YZ$.

Proof:

1. $X \rightarrow YZ$ (given)
2. $YZ \rightarrow Y$ (using IR1 Rule)
3. $X \rightarrow Y$ (using IR3 on 1 and 2)

IR6 (pseudotransitive rule): $\{X \rightarrow Y, WY \rightarrow Z\} \models WX \rightarrow Z$.

1. $X \rightarrow Y$ (given)
2. $WY \rightarrow Z$ (given)
3. $WX \rightarrow WY$ (using IR2 on 1 by augmenting with W)
4. $WX \rightarrow Z$ (using IR3 on 3 and 2)

22. Explain Equivalence sets of Functional dependencies.

A set of functional dependencies (FD) F is said to cover another set of functional dependencies E if every FD in E is also in F closure; that is, if every dependency in E can be inferred from F .

Alternatively, we can say E is covered by F. Two sets of functional dependencies E and F are equivalent if $E^+ = F^+$. That is E is equivalent to F if E covers F and F covers E.

To determine whether F covers E we calculate X^+ with respect to F for each FD $X \rightarrow y$ in E and then check whether X^+ includes the attributes Y.

Example:

$R = (A, C, D, E, H)$

$F1 = \{A \rightarrow C, AC \rightarrow D, E \rightarrow AD, E \rightarrow H\}$,

$F2 = \{A \rightarrow CD, E \rightarrow AH\}$

Check whether F1 and F2 are equivalent or not?

Solution:

To check F1 covers F2 –

$A^+ = \{A, C, D\}$ contains C, D

$E^+ = \{A, D, E, H\}$ contains A, H

So F1 covers F2

To check F2 covers F1:

$A^+ = \{A, C, D\}$ contains C

$\{A, C\}^+ = \{A, C, D\}$ contains D

$E^+ = \{A, C, D, E, H\}$ contains A, D, H

So F2 covers F1.

Hence F1 and F2 are equivalent.

23. Minimal sets of Functional dependencies.

A minimal cover of a set of functional dependencies (FD) E is a minimal set of dependencies F that is equivalent to E.

The formal definition is: A set of FD F to be minimal if it satisfies the following conditions –

- Every dependency in F has a single attribute for its right-hand side.
- We cannot replace any dependency $X \rightarrow A$ in F with a dependency $Y \rightarrow A$, where Y is a proper subset of X, and still have a set of dependencies that is equivalent to F.
- We cannot remove any dependency from F and still have a set of dependencies that are equivalent to F.

Steps to Find Minimal Cover

(i) Split the right-hand attributes of all FDs.

(ii) Remove all redundant FDs.

(iii) Find the Extraneous attribute and remove it.

Example:

Minimize $\{A \rightarrow C, AC \rightarrow D, E \rightarrow H, E \rightarrow AD\}$

Step 1: $\{A \rightarrow C, AC \rightarrow D, E \rightarrow H, E \rightarrow A, E \rightarrow D\}$

Step 2: $\{A \rightarrow C, AC \rightarrow D, E \rightarrow H, E \rightarrow A\}$

Here Redundant FD: $\{E \rightarrow D\}$

Step 3: $\{AC \rightarrow D\}$

$\{A\}^+ = \{A, C\}$

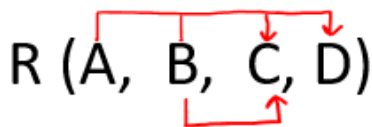
Therefore, C is extraneous and is removed.

$\{A \rightarrow D\}$

Minimal Cover = $\{A \rightarrow C, A \rightarrow D, E \rightarrow H, E \rightarrow A\}$

24. Given a relation R (A, B, C, D) and Functional Dependency set FD = { $AB \rightarrow CD$, $B \rightarrow C$ }, determine whether the given R is in 2NF? If not convert it into 2NF.

Let us construct an arrow diagram on R using FD to calculate the candidate key.



From above arrow diagram on R, we can see that an attributes AB is not determined by any of the given FD, hence AB will be the integral part of the Candidate key.

Let us calculate the closure of AB.

$AB^+ = ABCD$ (from closure method)

Since the closure of AB contains all the attributes of R, hence **AB is Candidate Key**.

Since R has 4 attributes: - A, B, C, D, and Candidate Key is AB, Therefore, prime attributes (part of candidate key) are A and B while a non-prime attribute are C and D.

a) FD: $AB \rightarrow CD$ satisfies the definition of 2NF, that non-prime attribute(C and D) are fully dependent on candidate key AB

b) FD: $B \rightarrow C$ does not satisfy the definition of 2NF, as a non-prime attribute(C) is partially dependent on candidate key AB(i.e. key should not be broken at any cost)

As FD $B \rightarrow C$, the above table R(A, B, C, D) is not in 2NF.

Convert the table R(A, B, C, D) in 2NF:

Since **FD: $B \rightarrow C$** , our table was not in 2NF, let's decompose the table

R1(B, C)

Since the key is AB, and from FD $AB \rightarrow CD$, we can create $R_2(A, B, C, D)$ but this will again have a problem of partial dependency $B \rightarrow C$, hence $R_2(A, B, D)$.

Finally, the decomposed table which is in 2NF

a) $R_1(B, C)$ b) $R_2(A, B, D)$

25. Given a relation $R(P, Q, R, S, T)$ and Functional Dependency set $FD = \{ PQ \rightarrow R, S \rightarrow T \}$, determine whether the given R is in 2NF? If not convert it into 2NF.

Let us construct an arrow diagram on R using FD to calculate the candidate key.



PQS will be the integral part of the Candidate key.

Since $PQS^+ = PQSRT$ (closure of PQS contains all the attributes of R)
So, **PQS is Candidate Key.**

Prime Attribute – PQS (because PQS is the candidate key)

Non prime Attribute – RT (because RT are not the part of the candidate key)

Given FD are $PQ \rightarrow R$ and $S \rightarrow T$

a) FD: $PQ \rightarrow R$ does not satisfy the definition of 2NF, that non-prime attribute(R) is partially dependent on part of candidate key PQS.

b) FD: $S \rightarrow T$ does not satisfy the definition of 2NF, as a non-prime attribute(T) is partially dependent on candidate key PQS (i.e., key should not be broken at any cost).

Hence, the above table $R(P, Q, R, S, T)$ is not in 2NF.

Convert the table $R(P, Q, R, S, T)$ in 2NF:

Since due to FD: $PQ \rightarrow R$ and $S \rightarrow T$, our table was not in 2NF, let's decompose the table

$R_1(P, Q, R)$ (Now in table R_1 FD: $PQ \rightarrow R$ is Full F D, hence R_1 is in 2NF)

$R_2(S, T)$ (Now in table R_2 FD: $S \rightarrow T$ is Full F D, hence R_2 is in 2NF)

And create one table for the key, since the key is PQS.

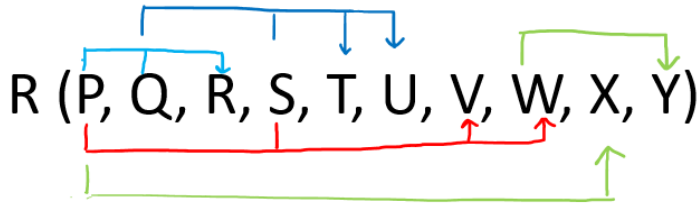
$R_3(P, Q, S)$

Finally, the decomposed tables which is in 2NF are:

a) **$R_1(P, Q, R)$** b) **$R_2(S, T)$** c) **$R_3(P, Q, S)$**

26. Given a relation R(P, Q, R, S, T, U, V, W, X, Y) and Functional Dependency set $FD = \{ PQ \rightarrow R, PS \rightarrow VW, QS \rightarrow TU, P \rightarrow X, W \rightarrow Y \}$, determine whether the given R is in 2NF? If not convert it into 2 NF.

Let us construct an arrow diagram on R using FD to calculate the candidate key.



PQS will be the integral part of the Candidate key.

Since, $PQS^+ = P Q S R T U V W X Y$ (closure of PQS contains all the attributes of R)
So, **PQS is Candidate Key.**

Prime Attribute – PQS (because PQS is the candidate key)

Non prime Attribute – RTUVWXY (not the part of the candidate key)

1. FD: $PQ \rightarrow R$ does not satisfy the definition of 2NF, that non-prime attribute(R) is partially dependent on part of candidate key PQS
2. FD: $PS \rightarrow VW$ does not satisfy the definition of 2NF, that non-prime attribute(VW) is partially dependent on part of candidate key PQS
3. FD: $QS \rightarrow TU$ does not satisfy the definition of 2NF, that non-prime attribute(TU) is partially dependent on part of candidate key PQS
4. FD: $P \rightarrow X$ does not satisfy the definition of 2NF, that non-prime attribute(X) are partially dependent on part of candidate key PQS
5. FD: $W \rightarrow Y$ does **not violate** the definition of 2NF, as the non-prime attribute(Y) is dependent on the non-prime attribute(W), which is not related to the definition of 2NF.

Hence because of FD: $PQ \rightarrow R, PS \rightarrow VW, QS \rightarrow TU, P \rightarrow X$ the above table R(P, Q, R, S, T, U, V, W, X, Y) is not in 2NF.

Convert the table R(P, Q, R, S, T, U, V, W, X, Y) in 2NF:

Since due to FD

: $PQ \rightarrow R, PS \rightarrow VW, QS \rightarrow TU, P \rightarrow X$ our table was not in 2NF, let's decompose the table

R1(P, Q, R) (Now in table R1 FD: $PQ \rightarrow R$ is Full F D, hence R1 is in 2NF)

R2(P, S, V, W) (Now in table R2 FD: $PS \rightarrow VW$ is Full F D, hence R2 is in 2NF)

R3(Q, S, T, U) (Now in table R3 FD: $QS \rightarrow TU$ is Full F D, hence R3 is in 2NF)

R4(P, X) (Now in table R4 FD : $P \rightarrow X$ is Full F D, hence R4 is in 2NF)

R5(W, Y) (Now in table R5 FD: $W \rightarrow Y$ is Full F D, hence R2 is in 2NF)

And create one table for the key, since the key is PQS.

R6(P, Q, S)

Finally, the decomposed tables which is in 2NF are:

R1(P, Q, R) R2(P, S, V, W) R3(Q, S, T, U)

R4(P, X) R5(W, Y) R6(P, Q, S)

27. Given a relation R(A, B, C, D, E) and Functional Dependency set $FD = \{ A \rightarrow B, B \rightarrow E, C \rightarrow D \}$, determine whether the given R is in 2NF? If not convert it into 2NF.

Solution:

First find out the candidate key of the relation.

Since $AC^+ = A C D B E$, (closure of AC contains all the attributes of R)

So **AC is candidate key** in above table.

Prime attribute - AC (because AC is the candidate key)

Non prime attribute - DBE (because DBE are not the part of the candidate key)

Now, the functional dependency $A \rightarrow B$ and $B \rightarrow E$ follow the rule of 2NF,

But **functional dependency $C \rightarrow D$ violates the rule of 2NF, because attribute C which is prime attribute (part of the candidate key AC) is determining the non prime attribute D.** It is partial dependency and this type of partial dependency is not allowed in 2NF.

Therefore, to convert the relation R(A B C D) in 2NF, It is divided into three relations:

R1 (A B E) -- [$A \rightarrow B, B \rightarrow E$]

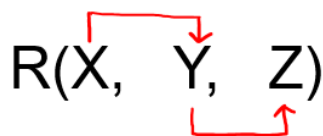
R2 (C D) -- [$C \rightarrow D$]

R3 (A C) -- [AC is candidate key]

Now R1, R2 and R3 are following the rules of 2NF.

28. Given a relation R(X, Y, Z) and Functional Dependency set $FD = \{ X \rightarrow Y \text{ and } Y \rightarrow Z \}$, determine whether the given R is in 3NF? If not convert it into 3NF.

Let us construct an arrow diagram on R using FD to calculate the candidate key.



X will be the integral part of the candidate key.

Since, $X^+ = XYZ$ (closure of X contains all the attributes of R)

So, **X is Candidate Key.**

Prime Attribute – X (because X is the candidate key)

Non prime Attribute – YZ (because YZ are not the part of the candidate key)

Given FD are $X \rightarrow Y$ and $Y \rightarrow Z$. So, we can write $X \rightarrow Z$ (which is a transitive dependency)
Hence, the table is not in 3NF.

Now check the above table is in 2 NF.

FD: $X \rightarrow Y$ is in 2NF (as Key is not breaking and its Fully functional dependent)

FD: $Y \rightarrow Z$ is also in 2NF(as it does not violate the definition of 2NF)

Convert the table R(X, Y, Z) into 3NF:

Since due to FD: $Y \rightarrow Z$, our table was not in 3NF, let's decompose the table

FD: $Y \rightarrow Z$ was creating issue, hence one table R1(Y, Z).

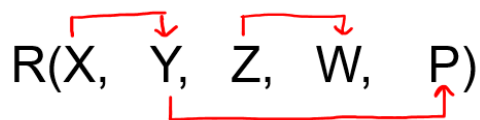
Create one Table for key X, R2(X, Y), since $X \rightarrow Y$.

Hence decomposed tables which are in 3NF are:

R1(X, Y) R2(Y, Z)

29. Given a relation R(X, Y, Z, W, P) and Functional Dependency set $FD = \{ X \rightarrow Y, Y \rightarrow P, \text{ and } Z \rightarrow W \}$, determine whether the given R is in 3NF? If not convert it into 3NF.

Let us construct an arrow diagram on R using FD to calculate the candidate key.



XZ will be the integral part of the candidate key.

Since, $XZ^+ = XZYWP$ (closure of XZ contains all the attributes of R)

So, **XZ is Candidate Key.**

Prime Attribute – XZ (because XZ is the candidate key)

Non prime Attribute – YWP (because YWP are not the part of the candidate key)

Given FD are $X \rightarrow Y$, $Y \rightarrow P$, and $Z \rightarrow W$ and Super Key / Candidate Key is XZ

1. FD: $X \rightarrow Y$ does not satisfy the definition of 3NF, that neither X is Super Key nor Y is a prime attribute.
2. FD: $Y \rightarrow P$ does not satisfy the definition of 3NF, that neither Y is Super Key nor P is a prime attribute.
3. FD: $Z \rightarrow W$ satisfies the definition of 3NF, that neither Z is Super Key nor W is a prime attribute.

Convert the table R(X, Y, Z, W, P) into 3NF:

Since all the FD = { $X \rightarrow Y$, $Y \rightarrow P$, and $Z \rightarrow W$ } were not in 3NF, let us convert R in 3NF

R1(X, Y) {Using FD $X \rightarrow Y$ }

R2(Y, P) {Using FD $Y \rightarrow P$ }

R3(Z, W) {Using FD $Z \rightarrow W$ }

And create one table for Candidate Key XZ

R4(X, Z) { Using Candidate Key XZ }

All the decomposed tables R1, R2, R3, and R4 are in 2NF(as there is no partial dependency) as well as in 3NF.

Hence decomposed tables are:

R1(X, Y) R2(Y, P) R3(Z, W) R4(X, Z)